Integration Manual

for S32G PFE MCAL 4.4 Driver

Document Number: IM11PFEASR4.4 Rev0000R1.4.0 Rev. 2.5

Chapter 1

Revision History

Revision	Date	Description
1.0	2021-04-21	Prepared for release of S32G PFE MCAL 4.4 Driver Version 0.9.0
1.1	2021-07-01	Prepared for release of S32G PFE MCAL 4.4 Driver Version 0.9.4
1.2	2021-10-12	Prepared for release of S32G PFE MCAL 4.4 Driver Version 0.9.5
1.3	2021-12-22	Prepared for release of S32G PFE MCAL 4.4 Driver Version 0.9.6
1.4	2022-02-04	Prepared for release of S32G PFE MCAL 4.4 Driver Version 0.9.7-CD01
1.5	2022-02-10	Prepared for release of S32G PFE MCAL 4.4 Driver Version 0.9.7-CD02
1.6	2022-04-07	Prepared for release of S32G PFE MCAL 4.4 Driver Version 0.9.7
1.7	2022-07-27	Prepared for release of S32G PFE MCAL 4.4 Driver Version 0.9.8
1.8	2022-10-12	Prepared for release of S32G PFE MCAL 4.4 Driver Version 1.0.0_CD1
1.9	2022-11-15	Prepared for release of S32G PFE MCAL 4.4 Driver Version 1.0.0
2.0	2023-05-15	Prepared for release of S32G PFE MCAL 4.4 Driver Version 1.1.0_CD2
2.1	2023-05-22	Prepared for release of S32G PFE MCAL 4.4 Driver Version 1.1.0
2.2	2023-07-27	Prepared for release of S32G PFE MCAL 4.4 Driver Version 1.2.0
2.3	2023-11-03	Prepared for release of S32G PFE MCAL 4.4 Driver Version 1.3.0_CD1
2.4	2023-12-15	Added documentation for errata, warnings and new features, compiler options documentation synchronized with RTD project, updated for driver version 1.3.0
2.5	2024-05-28	Prepared for release of S32G PFE MCAL 4.4 Driver Version 1.4.0

Chapter 2

Introduction

This Integration Manual describes the integration requirements for NXP Semiconductors' AUTOSAR Ethernet Driver for PFE on S32XX.

2.1 Supported Derivatives

The software described in this document is intended to be used with the following microcontroller devices of NXP Semiconductors:

- S32G2XX, revision 2.0 and revision 2.1:
  - S32G274A_BGA525
  - S32G254A_BGA525
  - S32G233A_BGA525
  - S32G234M_BGA525
- S32G3XX, revision 1.1:
  - S32G378A_BGA525
  - S32G379A_BGA525
  - S32G398A_BGA525
  - S32G399A_BGA525
  - S32G338M_BGA525
  - S32G339M_BGA525
  - S32G358A_BGA525
  - S32G359A_BGA525

2.2 Overview

AUTOSAR (AUTomotive Open System ARchitecture) is an industry partnership working to establish standards for software interfaces and software modules for automobile electronic control systems.

AUTOSAR:

- paves the way for innovative electronic systems that further improve performance, safety and environmental friendliness.
- is a strong global partnership that creates one common standard: "Cooperate on standards, compete on implementation".
- is a key enabling technology to manage the growing electrics/electronics complexity. It aims to be prepared for the upcoming technologies and to improve cost-efficiency without making any compromise with respect to quality.
- facilitates the exchange and update of software and hardware over the service life of the vehicle.

2.3 About This Manual

This Technical Reference employs the following typographical conventions:

- **Boldface** style: Used for important terms, notes and warnings.
- *Italic* style: Used for code snippets in the text. Note that C language modifiers such "const" or "volatile" are sometimes omitted to improve readability of the presented code.

Notes and warnings are shown as below:

Note

This is a note.

Warning

This is a warning

2.4 Acronyms and Definitions

Term	Definition
API	Application Programming Interface
AUTOSAR	Automotive Open System Architecture
DEM	Diagnostic Event Manager
DET	Default Error Tracer
ETH	Ethernet
ETHIF	Ethernet Interface
ETHTRCV	Ethernet Transceiver
ETHSWT	Ethernet Switch
HIF	Host InterFace
ISR	Interrupt Service Routine
MCU	Micro Controller Unit
MII	Media Independent Interface
N/A	Not Available
PFE	Packet Forwarding Engine
PHY	PHYsical layer
RMII	Reduced Media Independent Interface
RAM	Random Access Memory
IDEX	Inter Driver EXchange

- The term "PFE" is related to the hardware module providing the Ethernet functionality.
- The term "Ethernet Controller" is related to a software interface providing standardized access to PFE.
- The term "Ethernet Driver" is related to the software handling the Ethernet Controllers and PFE.
- The term "Application" is used for the software utilizing the Ethernet Driver.
- The term "IDEX" is a request-response based communication protocol between PFE Driver instances.

2.5 Reference List

#	Title	Version
1	Specification of Ethernet Driver	AUTOSAR Release 4.↩ 4.0
2	Specification of Ethernet Interface	AUTOSAR Release 4.↩ 4.0
3	Requirements on Ethernet Support in AUTOSAR	AUTOSAR Release 4.↩ 4.0
4	S32G2 Reference Manual	Rev. 7, 02/2023
5	S32G3 Reference Manual	Rev. 4, 10/2023
6	FCI API Reference	Rev. 2.5.0
7	User Manual for S32 BASENXP Driver	Rev0000R4.0.2 Rev. 1.0
8	Integration Manual for S32 BASENXP Driver	Rev0000R4.0.2 Rev. 1.0
9	User Manual for S32 PLATFORM Driver	Rev0000R4.0.0 Rev. 1.0

#	Title	Version
10	Integration Manual for S32 PLATFORM Driver	Rev0000R4.0.0 Rev. 1.0
11	User Manual for S32 MCU Driver	Rev0000R4.0.0 Rev. 1.0
12	Integration Manual for S32 MCU Driver	Rev0000R4.0.0 Rev. 1.0
13	User Manual for S32 PORT Driver	Rev0000R4.0.0 Rev. 1.0
14	Integration Manual for S32 PORT Driver	Rev0000R4.0.0 Rev. 1.0
15	User Manual for S32 SERDES Driver	Rev0000R4.0.0 Rev. 1.0
16	Integration Manual for S32 SERDES Driver	Rev0000R4.0.0 Rev. 1.0
17	PFE Health Monitoring	Rev. 1.4.1, 12/2023

Chapter 3

Installing the driver

This section describes two ways to install the PFE MCAL plugin into the EB Tresos Studio.

3.1 Installation Directly in Tresos Plugins Folder

The PFE MCAL plugin can be found in release package in folder eclipse/plugins, its name starts with Eth_43_P↩ FE_TS_. To install the plugin, copy it from release package to Tresos *plugins* folder.

3.2 Installation Through a Link

1. For this method it is necessary to extract the release package to a location accessible by EB Tresos Studio, e.g. C:\NXP
2. Next step is to create a link file in *links* folder in Tresos installation folder. The filename extension shall be *link* and for start it can be an empty file, e.g. C:\EB\tresos\links\PFE-DRV_S32G_M7_MCAL_1.1.0.link
3. At last write the link in the link file. The syntax is: path=path_to_release_package, where the path_to_↩ release_package is path in Windows format with forward slashes, e.g. path=c:/NXP/PFE-DRV_S32G_M7↩ _MCAL_1.1.0
4. The link file can be deleted when the driver is no longer needed. It is necessary to delete links to any potential prior codedrops as they may create conflicts, if they contain plugin with same name.

Chapter 4

Building the driver

- Build Options
- Files required for compilation
- Setting up the plugins
- Memory Layout Control

This section describes the source files and various compilers, linker options used for building the driver.

It also explains the EB Tresos Studio plugin setup procedure.

4.1 Build Options

- GCC Compiler/Assembler/Linker Options
- GHS Compiler/Assembler/Linker Options
- DIAB Compiler/Assembler/Linker Options

The PFE driver files are compiled using:

- NXP GCC 9.2.0 20190812 (Build 1649 Revision gaf57174)
- Green Hills Multi 7.1.6d / Compiler 2020.1.4
- Wind River Diab Compiler 7.0.3

The compiler, assembler, and linker flags used for building the driver are explained below.

The TS_T40D11M14I0R0 part of the plugin name is composed as follows:

- T = Target_Id (e.g. T40 identifies Cortex-M architecture)
- D = Derivative_Id (e.g. D11 identifies S32 platform)
- M = SW_Version_Major and SW_Version_Minor
- I = SW_Version_Patch
- R = Reserved

4.1.1 GCC Compiler/Assembler/Linker Options

4.1.1.1 GCC Compiler Options

Compiler Option	Description
-mcpu=cortex-m7	Targeted ARM processor for which GCC should tune the performance of the code
-mthumb	Generates code that executes in Thumb state
-mlittle-endian	Generate code for a processor running in little-endian mode
-mfpu=fpv5-sp-d16	Specifies the floating-point hardware available on the target
-mfloat-abi=hard	Specifies the floating-point ABI to use. "hard" allows generation of floating-point instructions and uses FPU-specific calling conventions
-std=c99	Specifies the ISO C99 base standard
-Os	Optimize for size. Enables all -O2 optimizations except those that often increase code size
-ggdb3	Produce debugging information for use by GDB using the most expressive format available, including GDB extensions if at all possible. Level 3 includes extra information, such as all the macro definitions present in the program
-Wall	Enables all the warnings about constructions that some users consider questionable, and that are easy to avoid (or modify to prevent the warning), even in conjunction with macros
-Wextra	This enables some extra warning flags that are not enabled by -Wall
-pedantic	Issue all the warnings demanded by strict ISO C. Reject all programs that use forbidden extensions. Follows the version of the ISO C standard specified by the aforementioend -std option
-Wstrict-prototypes	Warn if a function is declared or defined without specifying the argument types
-Wundef	Warn if an undefined identifier is evaluated in an #if directive. Such identifiers are replaced with zero
-Wunused	Warn whenever a function, variable, label, value, macro is unused
-Werror=implicit-function-declaration	Make the specified warning into an error. This option throws an error when a function is used before being declared
-Wsign-compare	Warn when a comparison between signed and unsigned values could produce an incorrect result when the signed value is converted to unsigned.
-Wdouble-promotion	Give a warning when a value of type float is implicitly promoted to double
-fno-short-enums	Specifies that the size of an enumeration type is at least 32 bits regardless of the size of the enumerator values.
-funsigned-char	Let the type char be unsigned by default, when the declaration does not use either signed or unsigned
-funsigned-bitfields	Let a bit-field be unsigned by default, when the declaration does not use either signed or unsigned

Compiler Option	Description
-fomit-frame-pointer	Omit the frame pointer in functions that don't need one. This avoids the instructions to save, set up and restore the frame pointer; on many targets it also makes an extra register available.
-fno-common	Makes the compiler place uninitialized global variables in the BSS section of the object file. This inhibits the merging of tentative definitions by the linker so you get a multiple-definition error if the same variable is accidentally defined in more than one compilation unit
-fstack-usage	Makes the compiler output stack usage information for the program, on a per-function basis
-fdump-ipa-all	Enables all inter-procedural analysis dumps
-c	Stop after assembly and produce an object file for each source file
-DS32XX	Predefine S32XX as a macro, with definition 1
-DS32G2XX	Predefine S32G2XX as a macro, with definition 1
-DS32G3XX	Predefine S32G3XX as a macro, with definition 1
-DGCC	Predefine GCC as a macro, with definition 1
-DUSE_SW_VECTOR_MODE	Predefine USE_SW_VECTOR_MODE as a macro, with definition 1. By default, the drivers are compiled to handle interrupts in Software Vector Mode
-DD_CACHE_ENABLE	Predefine D_CACHE_ENABLE as a macro, with definition 1. Enables data cache initalization in source file system.↩ c under the Platform driver
-DI_CACHE_ENABLE	Predefine I_CACHE_ENABLE as a macro, with definition 1. Enables instruction cache initalization in source file system.c under the Platform driver
-DENABLE_FPU	Predefine ENABLE_FPU as a macro, with definition 1. Enables FPU initalization in source file system.c under the Platform driver
-DMCAL_ENABLE_USER_MODE_SUPPORT	Predefine MCAL_ENABLE_USER_MODE_SUPPO↩ RT as a macro, with definition 1. Allows drivers to be configured in user mode.

4.1.1.2 GCC Assembler Options

Assembler Option	Description
-x assembler-with-cpp	Specifies the language for the following input files (rather than letting the compiler choose a default based on the file name suffix)
-mcpu=cortex-m7	Targeted ARM processor for which GCC should tune the performance of the code
-mfpu=fpv5-sp-d16	Specifies the floating-point hardware available on the target
-mfloat-abi=hard	Specifies the floating-point ABI to use. "hard" allows generation of floating-point instructions and uses FPU-specific calling conventions
-mthumb	Generates code that executes in Thumb state
-c	Stop after assembly and produce an object file for each source file

4.1.1.3 GCC Linker Options

Linker Option	Description
-Wl,-Map,filename	Produces a map file
-T linkerfile	Use linkerfile as the linker script. This script replaces the default linker script (rather than adding to it)
–entry=Reset_Handler	Specifies that the program entry point is Reset_Handler
-nostartfiles	Do not use the standard system startup files when linking
-mcpu=cortex-m7	Targeted ARM processor for which GCC should tune the performance of the code
-mthumb	Generates code that executes in Thumb state
-mfpu=fpv5-sp-d16	Specifies the floating-point hardware available on the target
-mfloat-abi=hard	Specifies the floating-point ABI to use. "hard" allows generation of floating-point instructions and uses FPU-specific calling conventions
-mlittle-endian	Generate code for a processor running in little-endian mode
-ggdb3	Produce debugging information for use by GDB using the most expressive format available, including GDB extensions if at all possible. Level 3 includes extra information, such as all the macro definitions present in the program
-lc	Link with the C library
-lm	Link with the Math library
-lgcc	Link with the GCC library

4.1.1.4 Known issue with GCC compiler

Warning regarding enum size is thrown by the linker due to usage of "-fno-short-enums" option: "(...) uses variable-size enums yet the output is to use 32-bit enums; use of enum values across objects may fail". The driver do not use any library enum types and the 3 reported library functions do not use any enums in their API - no functional impact.

4.1.2 GHS Compiler/Assembler/Linker Options

4.1.2.1 GHS Compiler Options

Compiler Option	Description
-cpu=cortexm7	Selects target processor: Arm Cortex M7
-thumb	Selects generating code that executes in Thumb state
-fpu=vfpv5_d16	Specifies hardware floating-point using the v5 version of the VFP instruction set, with 16 double-precision floating-point registers
-fsingle	Use hardware single-precision, software double-precision FP instructions
-C99	Use (strict ISO) C99 standard (without extensions)
–ghstd=last	Use the most recent version of Green Hills Standard mode (which enables warnings and errors that enforce a stricter coding standard than regular C and C++)

Compiler Option	Description
-Osize	Optimize for size
–gnu_asm	Enables GNU extended asm syntax support
-dual_debug	Generate DWARF 2.0 debug information
-G	Generate debug information
-keeptempfiles	Prevents the deletion of temporary files after they are used. If an assembly language file is created by the compiler, this option will place it in the current directory instead of the temporary directory
-Wimplicit-int	Produce warnings if functions are assumed to return int
-Wshadow	Produce warnings if variables are shadowed
-Wtrigraphs	Produce warnings if trigraphs are detected
-Wundef	Produce a warning if undefined identifiers are used in #if preprocessor statements
–unsigned_chars	Let the type char be unsigned, like unsigned char
–unsigned_fields	Bitfelds declared with an integer type are unsigned
–no_commons	Allocates uninitialized global variables to a section and initializes them to zero at program startup
–no_exceptions	Disables C++ support for exception handling
–no_slash_comment	C++ style // comments are not accepted andgenerate errors
–prototype_errors	Controls the treatment of functions referenced or called when no prototype has been provided
–incorrect_pragma_warnings	Controls the treatment of valid #pragma directives that use the wrong syntax
-c	Stop after assembly and produce an object file for each source file
-DS32XX	Predefine S32XX as a macro, with definition 1
-DS32G2XX	Predefine S32G2XX as a macro, with definition 1
-DS32G3XX	Predefine S32G3XX as a macro, with definition 1
-DGHS	Predefine GHS as a macro, with definition 1
-DUSE_SW_VECTOR_MODE	Predefine USE_SW_VECTOR_MODE as a macro, with definition 1. By default, the drivers are compiled to handle interrupts in Software Vector Mode
-DD_CACHE_ENABLE	Predefine D_CACHE_ENABLE as a macro, with definition 1. Enables data cache initalization in source file system.↩ c under the Platform driver
-DI_CACHE_ENABLE	Predefine I_CACHE_ENABLE as a macro, with definition 1. Enables instruction cache initalization in source file system.c under the Platform driver
-DENABLE_FPU	Predefine ENABLE_FPU as a macro, with definition 1. Enables FPU initalization in source file system.c under the Platform driver
-DMCAL_ENABLE_USER_MODE_SUPPORT	Predefine MCAL_ENABLE_USER_MODE_SUPPO↩ RT as a macro, with definition 1. Allows drivers to be configured in user mode

4.1.2.2 GHS Assembler Options

Assembler Option	Description
-cpu=cortexm7	Selects target processor: Arm Cortex M7
-fpu=vfpv5_d16	Specifies hardware floating-point using the v5 version of the VFP instruction set, with 16 double-precision floating-point registers
-fsingle	Use hardware single-precision, software double-precision FP instructions
-preprocess_assembly_files	Controls whether assembly files with standard extensions such as .s and .asm are preprocessed
-list	Creates a listing by using the name and directory of the object file with the .lst extension
-c	Stop after assembly and produce an object file for each source file

4.1.2.3 GHS Linker Options

Linker Option	Description
-e Reset_Handler	Make the symbol Reset_Handler be treated as a root symbol and the start label of the application
-T linker_script_file.ld	Use linker_script_file.ld as the linker script. This script replaces the default linker script (rather than adding to it)
-map	Produce a map file
-keepmap	Controls the retention of the map file in the event of a link error
-Mn	Generates a listing of symbols sorted alphabetically/numerically by address
-delete	Instructs the linker to remove functions that are not referenced in the final executable. The linker iterates to find functions that do not have relocations pointing to them and eliminates them
-ignore_debug_references	Ignores relocations from DWARF debug sections when using -delete. DWARF debug information will contain references to deleted functions that may break some third-party debuggers
-Llibrary_path	Points to library_path (the libraries location) for thumb2 to be used for linking
-larch	Link architecture specific library
-lstartup	Link run-time environment startup routines. The source code for themodules in this library is provided in the src/libstartup directory
-lind_sd	Link language-independent library, containing support routines for features such as software floating point, run-time error checking, C99 complex numbers, and some general purpose routines of the ANSI C library
-v	Prints verbose information about the activities of the linker, including the libraries it searches to resolve undefined symbols
-nostartfiles	Controls the start files to be linked into the executable

4.1.3 DIAB Compiler/Assembler/Linker Options

4.1.3.1 DIAB Compiler Options

Compiler Option	Description
-tARMCORTEXM7MG:simple	Selects target processor (hardware single-precision, software double-precision floating-point)

Compiler Option	Description
-mthumb	Selects generating code that executes in Thumb state
-std=c99	Follows the C99 standard for C
-Oz	Like -O2 with further optimizations to reduce code size
-g	Generates DWARF 4.0 debug information
-fstandalone-debug	Emits full debug info for all types used by the program
-Wstrict-prototypes	Warn if a function is declared or defined without specifying the argument types
-Wsign-compare	Produce warnings when comparing signed type with unsigned type
-Wdouble-promotion	Give a warning when a value of type float is implicitly promoted to double
-Wunknown-pragmas	Issues a warning for unknown pragmas
-Wundef	Warns if an undefined identifier is evaluated in an #if directive. Such identifiers are replaced with zero
-Wextra	Enables some extra warning flags that are not enabled by '-Wall'
-Wall	Enables all of the most useful warnings (for historical reasons this option does not literally enable all warnings)
-pedantic	Emits a warning whenever the standard specified by the -std option requires a diagnostic
-Werror=implicit-function-declaration	Generates an error whenever a function is used before being declared
-fno-common	Compile common globals like normal definitions
-fno-signed-char	Char is unsigned
-fno-trigraphs	Do not process trigraph sequences
-V	Displays the current version number of the tool suite
-c	Stop after assembly and produce an object file for each source file
-DS32XX	Predefine S32XX as a macro, with definition 1
-DS32G2XX	Predefine S32G2XX as a macro, with definition 1
-DS32G3XX	Predefine S32G3XX as a macro, with definition 1
-DDIAB	Predefine DIAB as a macro, with definition 1
-DUSE_SW_VECTOR_MODE	Predefine USE_SW_VECTOR_MODE as a macro, with definition 1. By default, the drivers are compiled to handle interrupts in Software Vector Mode
-DD_CACHE_ENABLE	Predefine D_CACHE_ENABLE as a macro, with definition 1. Enables data cache initalization in source file system.↩ c under the Platform driver
-DI_CACHE_ENABLE	Predefine I_CACHE_ENABLE as a macro, with definition 1. Enables instruction cache initalization in source file system.c under the Platform driver
-DENABLE_FPU	Predefine ENABLE_FPU as a macro, with definition 1. Enables FPU initalization in source file system.c under the Platform driver
-DMCAL_ENABLE_USER_MODE_SUPPORT	Predefine MCAL_ENABLE_USER_MODE_SUPPO↩ RT as a macro, with definition 1. Allows drivers to be configured in user mode

4.1.3.2 DIAB Assembler Options

Assembler Option	Description
-mthumb	Selects generating code that executes in Thumb state
-Xpreprocess-assembly	Invokes C preprocessor on assembly files before running the assembler
-Xassembly-listing	Produces an .lst assembly listing file
-c	Stop after assembly and produce an object file for each source file

4.1.3.3 DIAB Linker Options

Linker Option	Description
-e Reset_Handler	Make the symbol Reset_Handler be treated as a root symbol and the start label of the application
linker_script_file.dld	Use linker_script_file.dld as the linker script. This script replaces the default linker script (rather than adding to it)
-m30	m2 + m4 + m8 + m16
-Xstack-usage	Gathers and display stack usage at link time
-Xpreprocess-lecl	Perform pre-processing on linker scripts
-Llibrary_path	Points to the libraries location for ARMV7EMMG to be used for linking
-lc	Links with the standard C library
-lm	Links with the math library

4.2 Files required for compilation

This section describes the include files required to compile, assemble and link the PFE Ethernet Driver for S32G microcontrollers.

To avoid integration of incompatible files, all the include files from other modules shall have the same AR_MAJO↩ R_VERSION and AR_MINOR_VERSION, i.e. only files with the same AUTOSAR major and minor versions can be compiled.

4.2.1 Ethernet Driver Files:

- Eth_43_PFE_TS_T40D11M14I0R0/src/autolibc.c
- Eth_43_PFE_TS_T40D11M14I0R0/src/blalloc.c
- Eth_43_PFE_TS_T40D11M14I0R0/src/elf.c
- Eth_43_PFE_TS_T40D11M14I0R0/src/Eth_43_PFE.c
- Eth_43_PFE_TS_T40D11M14I0R0/src/Eth_43_PFE_Irq.c
- Eth_43_PFE_TS_T40D11M14I0R0/src/Eth_PFE_LLD.c
- Eth_43_PFE_TS_T40D11M14I0R0/src/fci.c

- Eth_43_PFE_TS_T40D11M14I0R0/src/fci_connections.c
- Eth_43_PFE_TS_T40D11M14I0R0/src/fci_core_autosar.c
- Eth_43_PFE_TS_T40D11M14I0R0/src/fci_fp.c
- Eth_43_PFE_TS_T40D11M14I0R0/src/fci_fp_db.c
- Eth_43_PFE_TS_T40D11M14I0R0/src/fci_fw_features.c
- Eth_43_PFE_TS_T40D11M14I0R0/src/fci_hm.c
- Eth_43_PFE_TS_T40D11M14I0R0/src/fci_interfaces.c
- Eth_43_PFE_TS_T40D11M14I0R0/src/fci_l2br.c
- Eth_43_PFE_TS_T40D11M14I0R0/src/fci_l2br_domains.c
- Eth_43_PFE_TS_T40D11M14I0R0/src/fci_mirror.c
- Eth_43_PFE_TS_T40D11M14I0R0/src/fci_owner.c
- Eth_43_PFE_TS_T40D11M14I0R0/src/fci_qos.c
- Eth_43_PFE_TS_T40D11M14I0R0/src/fci_routes.c
- Eth_43_PFE_TS_T40D11M14I0R0/src/fci_rt_db.c
- Eth_43_PFE_TS_T40D11M14I0R0/src/fci_timer_owner.c
- Eth_43_PFE_TS_T40D11M14I0R0/src/fifo.c
- Eth_43_PFE_TS_T40D11M14I0R0/src/isa.c
- Eth_43_PFE_TS_T40D11M14I0R0/src/libfci_autosar.c
- Eth_43_PFE_TS_T40D11M14I0R0/src/nxp_snprintf.c
- Eth_43_PFE_TS_T40D11M14I0R0/src/oal_irq_autosar.c
- Eth_43_PFE_TS_T40D11M14I0R0/src/oal_job_autosar.c
- Eth_43_PFE_TS_T40D11M14I0R0/src/oal_mm_autosar.c
- Eth_43_PFE_TS_T40D11M14I0R0/src/oal_time_autosar.c
- Eth_43_PFE_TS_T40D11M14I0R0/src/oal_util_autosar.c
- Eth_43_PFE_TS_T40D11M14I0R0/src/oal_util_net_autosar.c
- Eth_43_PFE_TS_T40D11M14I0R0/src/pfe_bmu.c
- Eth_43_PFE_TS_T40D11M14I0R0/src/pfe_bmu_csr.c
- Eth_43_PFE_TS_T40D11M14I0R0/src/pfe_bus_err.c
- Eth_43_PFE_TS_T40D11M14I0R0/src/pfe_bus_err_csr.c
- Eth_43_PFE_TS_T40D11M14I0R0/src/pfe_class.c
- Eth_43_PFE_TS_T40D11M14I0R0/src/pfe_class_csr.c
- Eth_43_PFE_TS_T40D11M14I0R0/src/pfe_ecc_err.c
- Eth_43_PFE_TS_T40D11M14I0R0/src/pfe_ecc_err_csr.c

- Eth_43_PFE_TS_T40D11M14I0R0/src/pfe_emac.c
- Eth_43_PFE_TS_T40D11M14I0R0/src/pfe_emac_csr.c
- Eth_43_PFE_TS_T40D11M14I0R0/src/pfe_emac_slave.c
- Eth_43_PFE_TS_T40D11M14I0R0/src/pfe_fail_stop.c
- Eth_43_PFE_TS_T40D11M14I0R0/src/pfe_fail_stop_csr.c
- Eth_43_PFE_TS_T40D11M14I0R0/src/pfe_feature_mgr.c
- Eth_43_PFE_TS_T40D11M14I0R0/src/pfe_fp.c
- Eth_43_PFE_TS_T40D11M14I0R0/src/pfe_fw_fail_stop.c
- Eth_43_PFE_TS_T40D11M14I0R0/src/pfe_fw_fail_stop_csr.c
- Eth_43_PFE_TS_T40D11M14I0R0/src/pfe_fw_feature.c
- Eth_43_PFE_TS_T40D11M14I0R0/src/pfe_gpi.c
- Eth_43_PFE_TS_T40D11M14I0R0/src/pfe_gpi_csr.c
- Eth_43_PFE_TS_T40D11M14I0R0/src/pfe_hif.c
- Eth_43_PFE_TS_T40D11M14I0R0/src/pfe_hif_chnl.c
- Eth_43_PFE_TS_T40D11M14I0R0/src/pfe_hif_csr.c
- Eth_43_PFE_TS_T40D11M14I0R0/src/pfe_hif_drv.c
- Eth_43_PFE_TS_T40D11M14I0R0/src/pfe_hif_nocpy.c
- Eth_43_PFE_TS_T40D11M14I0R0/src/pfe_hif_nocpy_csr.c
- Eth_43_PFE_TS_T40D11M14I0R0/src/pfe_hif_ptp.c
- Eth_43_PFE_TS_T40D11M14I0R0/src/pfe_hif_ring.c
- Eth_43_PFE_TS_T40D11M14I0R0/src/pfe_hm.c
- Eth_43_PFE_TS_T40D11M14I0R0/src/pfe_host_fail_stop.c
- Eth_43_PFE_TS_T40D11M14I0R0/src/pfe_host_fail_stop_csr.c
- Eth_43_PFE_TS_T40D11M14I0R0/src/pfe_hw_feature.c
- Eth_43_PFE_TS_T40D11M14I0R0/src/pfe_idex.c
- Eth_43_PFE_TS_T40D11M14I0R0/src/pfe_if_db.c
- Eth_43_PFE_TS_T40D11M14I0R0/src/pfe_l2br.c
- Eth_43_PFE_TS_T40D11M14I0R0/src/pfe_l2br_table.c
- Eth_43_PFE_TS_T40D11M14I0R0/src/pfe_log_if.c
- Eth_43_PFE_TS_T40D11M14I0R0/src/pfe_mac_db.c
- Eth_43_PFE_TS_T40D11M14I0R0/src/pfe_minihif_drv.c
- Eth_43_PFE_TS_T40D11M14I0R0/src/pfe_mirror.c
- Eth_43_PFE_TS_T40D11M14I0R0/src/pfe_parity.c

- Eth_43_PFE_TS_T40D11M14I0R0/src/pfe_parity_csr.c
- Eth_43_PFE_TS_T40D11M14I0R0/src/pfe_pe.c
- Eth_43_PFE_TS_T40D11M14I0R0/src/pfe_phy_if.c
- Eth_43_PFE_TS_T40D11M14I0R0/src/pfe_phy_if_slave.c
- Eth_43_PFE_TS_T40D11M14I0R0/src/pfe_platform_master.c
- Eth_43_PFE_TS_T40D11M14I0R0/src/pfe_platform_slave.c
- Eth_43_PFE_TS_T40D11M14I0R0/src/pfe_rtable.c
- Eth_43_PFE_TS_T40D11M14I0R0/src/pfe_tmu.c
- Eth_43_PFE_TS_T40D11M14I0R0/src/pfe_tmu_csr.c
- Eth_43_PFE_TS_T40D11M14I0R0/src/pfe_util.c
- Eth_43_PFE_TS_T40D11M14I0R0/src/pfe_util_csr.c
- Eth_43_PFE_TS_T40D11M14I0R0/src/pfe_wdt.c
- Eth_43_PFE_TS_T40D11M14I0R0/src/pfe_wdt_csr.c
- Eth_43_PFE_TS_T40D11M14I0R0/include/autolibc.h
- Eth_43_PFE_TS_T40D11M14I0R0/include/blalloc.h
- Eth_43_PFE_TS_T40D11M14I0R0/include/ct_assert.h
- Eth_43_PFE_TS_T40D11M14I0R0/include/elf.h
- Eth_43_PFE_TS_T40D11M14I0R0/include/elf_cfg.h
- Eth_43_PFE_TS_T40D11M14I0R0/include/Eth_43_PFE.h
- Eth_43_PFE_TS_T40D11M14I0R0/include/Eth_43_PFE_Irq.h
- Eth_43_PFE_TS_T40D11M14I0R0/include/Eth_PFE_LLD.h
- Eth_43_PFE_TS_T40D11M14I0R0/include/fci.h
- Eth_43_PFE_TS_T40D11M14I0R0/include/fci_core.h
- Eth_43_PFE_TS_T40D11M14I0R0/include/fci_fp.h
- Eth_43_PFE_TS_T40D11M14I0R0/include/fci_fp_db.h
- Eth_43_PFE_TS_T40D11M14I0R0/include/fci_fw_features.h
- Eth_43_PFE_TS_T40D11M14I0R0/include/fci_internal.h
- Eth_43_PFE_TS_T40D11M14I0R0/include/fci_mirror.h
- Eth_43_PFE_TS_T40D11M14I0R0/include/fci_msg.h
- Eth_43_PFE_TS_T40D11M14I0R0/include/fci_msg_autosar.h
- Eth_43_PFE_TS_T40D11M14I0R0/include/fci_ownership_mask.h
- Eth_43_PFE_TS_T40D11M14I0R0/include/fci_rt_db.h
- Eth_43_PFE_TS_T40D11M14I0R0/include/fifo.h

- Eth_43_PFE_TS_T40D11M14I0R0/include/fpp.h
- Eth_43_PFE_TS_T40D11M14I0R0/include/fpp_ext.h
- Eth_43_PFE_TS_T40D11M14I0R0/include/hal.h
- Eth_43_PFE_TS_T40D11M14I0R0/include/isa.h
- Eth_43_PFE_TS_T40D11M14I0R0/include/libfci.h
- Eth_43_PFE_TS_T40D11M14I0R0/include/linked_list.h
- Eth_43_PFE_TS_T40D11M14I0R0/include/nxp_snprintf.h
- Eth_43_PFE_TS_T40D11M14I0R0/include/nxp_snprintf_cfg.h
- Eth_43_PFE_TS_T40D11M14I0R0/include/oal.h
- Eth_43_PFE_TS_T40D11M14I0R0/include/oal_irq.h
- Eth_43_PFE_TS_T40D11M14I0R0/include/oal_irq_autosar.h
- Eth_43_PFE_TS_T40D11M14I0R0/include/oal_job.h
- Eth_43_PFE_TS_T40D11M14I0R0/include/oal_master_if.h
- Eth_43_PFE_TS_T40D11M14I0R0/include/oal_mm.h
- Eth_43_PFE_TS_T40D11M14I0R0/include/oal_mutex_autosar.h
- Eth_43_PFE_TS_T40D11M14I0R0/include/oal_sync.h
- Eth_43_PFE_TS_T40D11M14I0R0/include/oal_time.h
- Eth_43_PFE_TS_T40D11M14I0R0/include/oal_types.h
- Eth_43_PFE_TS_T40D11M14I0R0/include/oal_types_autosar.h
- Eth_43_PFE_TS_T40D11M14I0R0/include/oal_util.h
- Eth_43_PFE_TS_T40D11M14I0R0/include/oal_util_autosar.h
- Eth_43_PFE_TS_T40D11M14I0R0/include/oal_util_net.h
- Eth_43_PFE_TS_T40D11M14I0R0/include/oal_util_net_autosar.h
- Eth_43_PFE_TS_T40D11M14I0R0/include/pfe_bmu.h
- Eth_43_PFE_TS_T40D11M14I0R0/include/pfe_bmu_csr.h
- Eth_43_PFE_TS_T40D11M14I0R0/include/pfe_bus_err.h
- Eth_43_PFE_TS_T40D11M14I0R0/include/pfe_bus_err_csr.h
- Eth_43_PFE_TS_T40D11M14I0R0/include/pfe_cbus.h
- Eth_43_PFE_TS_T40D11M14I0R0/include/pfe_class.h
- Eth_43_PFE_TS_T40D11M14I0R0/include/pfe_class_csr.h
- Eth_43_PFE_TS_T40D11M14I0R0/include/pfe_compiler.h
- Eth_43_PFE_TS_T40D11M14I0R0/include/pfe_ct.h
- Eth_43_PFE_TS_T40D11M14I0R0/include/pfe_ct_comp.h

- Eth_43_PFE_TS_T40D11M14I0R0/include/pfe_ecc_err.h
- Eth_43_PFE_TS_T40D11M14I0R0/include/pfe_ecc_err_csr.h
- Eth_43_PFE_TS_T40D11M14I0R0/include/pfe_emac.h
- Eth_43_PFE_TS_T40D11M14I0R0/include/pfe_emac_csr.h
- Eth_43_PFE_TS_T40D11M14I0R0/include/pfe_fail_stop.h
- Eth_43_PFE_TS_T40D11M14I0R0/include/pfe_fail_stop_csr.h
- Eth_43_PFE_TS_T40D11M14I0R0/include/pfe_feature_mgr.h
- Eth_43_PFE_TS_T40D11M14I0R0/include/pfe_fp.h
- Eth_43_PFE_TS_T40D11M14I0R0/include/pfe_fw_fail_stop.h
- Eth_43_PFE_TS_T40D11M14I0R0/include/pfe_fw_fail_stop_csr.h
- Eth_43_PFE_TS_T40D11M14I0R0/include/pfe_fw_feature.h
- Eth_43_PFE_TS_T40D11M14I0R0/include/pfe_global_wsp.h
- Eth_43_PFE_TS_T40D11M14I0R0/include/pfe_gpi.h
- Eth_43_PFE_TS_T40D11M14I0R0/include/pfe_gpi_csr.h
- Eth_43_PFE_TS_T40D11M14I0R0/include/pfe_hif.h
- Eth_43_PFE_TS_T40D11M14I0R0/include/pfe_hif_chnl.h
- Eth_43_PFE_TS_T40D11M14I0R0/include/pfe_hif_csr.h
- Eth_43_PFE_TS_T40D11M14I0R0/include/pfe_hif_drv.h
- Eth_43_PFE_TS_T40D11M14I0R0/include/pfe_hif_nocpy.h
- Eth_43_PFE_TS_T40D11M14I0R0/include/pfe_hif_nocpy_csr.h
- Eth_43_PFE_TS_T40D11M14I0R0/include/pfe_hif_ptp.h
- Eth_43_PFE_TS_T40D11M14I0R0/include/pfe_hif_ring.h
- Eth_43_PFE_TS_T40D11M14I0R0/include/pfe_hm.h
- Eth_43_PFE_TS_T40D11M14I0R0/include/pfe_host_fail_stop.h
- Eth_43_PFE_TS_T40D11M14I0R0/include/pfe_host_fail_stop_csr.h
- Eth_43_PFE_TS_T40D11M14I0R0/include/pfe_hw_feature.h
- Eth_43_PFE_TS_T40D11M14I0R0/include/pfe_idex.h
- Eth_43_PFE_TS_T40D11M14I0R0/include/pfe_if_db.h
- Eth_43_PFE_TS_T40D11M14I0R0/include/pfe_l2br.h
- Eth_43_PFE_TS_T40D11M14I0R0/include/pfe_l2br_table.h
- Eth_43_PFE_TS_T40D11M14I0R0/include/pfe_l2br_table_csr.h
- Eth_43_PFE_TS_T40D11M14I0R0/include/pfe_log_if.h
- Eth_43_PFE_TS_T40D11M14I0R0/include/pfe_mac_db.h

- Eth_43_PFE_TS_T40D11M14I0R0/include/pfe_minihif_drv.h
- Eth_43_PFE_TS_T40D11M14I0R0/include/pfe_mirror.h
- Eth_43_PFE_TS_T40D11M14I0R0/include/pfe_parity.h
- Eth_43_PFE_TS_T40D11M14I0R0/include/pfe_parity_csr.h
- Eth_43_PFE_TS_T40D11M14I0R0/include/pfe_pe.h
- Eth_43_PFE_TS_T40D11M14I0R0/include/pfe_phy_if.h
- Eth_43_PFE_TS_T40D11M14I0R0/include/pfe_platform.h
- Eth_43_PFE_TS_T40D11M14I0R0/include/pfe_platform_cfg.h
- Eth_43_PFE_TS_T40D11M14I0R0/include/pfe_platform_rpc.h
- Eth_43_PFE_TS_T40D11M14I0R0/include/pfe_rtable.h
- Eth_43_PFE_TS_T40D11M14I0R0/include/pfe_tmu.h
- Eth_43_PFE_TS_T40D11M14I0R0/include/pfe_tmu_csr.h
- Eth_43_PFE_TS_T40D11M14I0R0/include/pfe_util.h
- Eth_43_PFE_TS_T40D11M14I0R0/include/pfe_util_csr.h
- Eth_43_PFE_TS_T40D11M14I0R0/include/pfe_wdt.h
- Eth_43_PFE_TS_T40D11M14I0R0/include/pfe_wdt_csr.h

4.2.2 Ethernet Driver Generated Files (must be generated by the user using a configuration tool):

- Eth_43_PFE_Cfg.c
- Eth_43_PFE_Cfg.h
- Eth_43_PFE_[VariantName]_PBcfg.c
- Eth_43_PFE_[VariantName]_PBcfg.h
- pfe_cfg.h

4.2.3 Base Files:

- BaseNXP_TS_T40D11M40I2R0/include/Eth_GeneralTypes.h
- BaseNXP_TS_T40D11M40I2R0/include/Mcal.h
- BaseNXP_TS_T40D11M40I2R0/include/Eth_43_PFE_MemMap.h
- BaseNXP_TS_T40D11M40I2R0/include/Platform_Types.h
- BaseNXP_TS_T40D11M40I2R0/include/Soc_Ips.h
- BaseNXP_TS_T40D11M40I2R0/include/Std_Types.h
- BaseNXP_TS_T40D11M40I2R0/include/OsIf.h
- BaseNXP_TS_T40D11M40I2R0/generate_PC/include/modules.h

4.2.4 DEM Files:

- Dem_TS_T40D11M40I2R0/include/Dem.h
- Dem_TS_T40D11M40I2R0/include/Dem_Types.h
- Dem_TS_T40D11M40I2R0/generate_PC/include/Dem_IntErrId.h
- Dem_TS_T40D11M40I2R0/src/Dem.c

4.2.5 DET Files:

- Det_TS_T40D11M40I2R0/include/Det.h
- Det_TS_T40D11M40I2R0/src/Det.c

4.2.6 RTE Files:

- Rte_TS_T40D11M40I2R0/include/SchM_Eth_43_PFE.h
- Rte_TS_T40D11M40I2R0/src/SchM_Eth_43_PFE.c

4.2.7 EthIf Files:

- EthIf_TS_T40D11M40I2R0/include/EthIf_Cbk.h
- EthIf_TS_T40D11M40I2R0/src/EthIf_Cbk.c

4.2.8 EthTrcv Files:

- EthTrcv_TS_T40D11M40I2R0/include/EthTrcv.h
- EthTrcv_TS_T40D11M40I2R0/src/EthTrcv.c

4.2.9 EthSwt Files:

- EthSwt_TS_T40D11M40I2R0/include/EthSwt.h
- EthSwt_TS_T40D11M40I2R0/src/EthSwt.c

4.2.10 Time Service Files:

- Tm.h

4.3 Setting up the plugins

The Ethernet Driver was designed to be configured by using the EB Tresos Studio (version 27.1.0 b200625-0900 or later)

4.3.1 Location of various files inside the Ethernet driver folder:

- VSMD (Vendor Specific Module Definition) file in EB Tresos Studio XDM format:
  - Eth_43_PFE_TS_T40D11M14I0R0/config/Eth_43_PFE.xdm
- VSMD (Vendor Specific Module Definition) file in AUTOSAR compliant EPD format:
  - Eth_43_PFE_TS_T40D11M14I0R0/autosar/Eth_43_PFE_s32gxx.epd
- Code Generation Templates
  - Eth_43_PFE_TS_T40D11M14I0R0/generate/include/Eth_43_PFE_Cfg.h
  - Eth_43_PFE_TS_T40D11M14I0R0/generate/include/Eth_43_PFE_PBcfg.h
  - Eth_43_PFE_TS_T40D11M14I0R0/generate/include/pfe_cfg.h
  - Eth_43_PFE_TS_T40D11M14I0R0/generate/src/Eth_43_PFE_Cfg.c
  - Eth_43_PFE_TS_T40D11M14I0R0/generate/src/Eth_43_PFE_PBcfg.c
- Code Generation Include Files
  - Eth_43_PFE_TS_T40D11M14I0R0/generate/Eth_43_PFE_Checks.m
  - Eth_43_PFE_TS_T40D11M14I0R0/generate/Eth_43_PFE_GetDemParameters.m
  - Eth_43_PFE_TS_T40D11M14I0R0/generate/Eth_43_PFE_GetPfeParameters.m
  - Eth_43_PFE_TS_T40D11M14I0R0/generate/Eth_43_PFE_VersionChecks.m

4.3.2 Steps to generate the configuration:

1. Add the driver plugin Eth_43_PFE_TS_T40D11M14I0R0 into configuration project.
2. Add other plugins into configuration project, refer to Other AUTOSAR modules - dependencies.
3. Set the desired Tresos Output location folder for the generated sources and header files.
4. Use the EB Tresos Studio GUI to modify ECU configuration parameters values.
5. Generate the configuration files

4.4 Memory Layout Control

The PFE, via its bus masters, requires access to various data structures stored in host memory. The critical data structures are allocated to dedicated memory sections based on Eth_43_PFE_MemMap.h file. Integrator can configure placement of the memory sections in linker definition file, to select appropriate position either to optimize performance (put data to faster memory) and/or tune the application from security/safety perspective (put data into dedicated container with restricted access permissions). Here is the list of data structures accessed from host memory by PFE:

4.4.1 PFE System Buffers

This is the memory pool where Ethernet frames are stored while they're being processed by the PFE. This memory is allocated by Master driver only.

In case when the driver is using HIF channels only, it is normally not touching this memory and thus the host CPU core does not need to access it.

In case when the driver uses HIFNOCPY channel the driver/CPU is required to access this data. Number of BMU2 buffers needs to be configured higher in this case, because it is also used for all Rx and Tx buffers.

The MemMap file allocates the pool in VAR_CLEARED_UNSPECIFIED_BMU_MEM memory, which is put into ".pfe_bmu_mem" section.

Size of the pool is affected by configuration option "VarEthBmu2BufCnt" and can be calculated as:

- SystemBuffersMemSize[B] = VarEthBmu2BufCnt ∗ 2048

Note

The PFE SystemBuffers region must reside within the PFE-accessible range 0x00020000 - 0xBFFFFFFF.

The PFE SystemBuffers region must be aligned to its size (if the region size is 0x40000, it must be aligned to 0x40000).

Option VarEthBmu2BufCnt value	Required alignment of section .pfe_bmu_mem
32	0x10000 (64 KiB)
64	0x20000 (128 KiB)
128	0x40000 (256 KiB)
256	0x80000 (512 KiB)
512	0x100000 (1 MiB)
1024	0x200000 (2 MiB)

4.4.2 Buffer Descriptors

HIF RX and TX buffer descriptor rings and related structures are stored within this memory area. Both PFE and the host CPU running the driver need r/w access to this region.

The MemMap file allocates the pool in VAR_CLEARED_UNSPECIFIED_BD_MEM memory, which is put into ".pfe_bd_mem" section.

The driver can run in various configurations. Region size is affected by following configuration options: "Multiple PFE drivers support (master-slave)", EthCtrlConfigEgressFifoBufTotal, EthCtrlConfigIngressFifoBufTotal and "Common HIF interface" the minimum region size can be determined as:

- BDMemSize[B] = (ETH_43_PFE_MAX_RXBD_CNT + ETH_43_PFE_MAX_TXBD_CNT) ∗ MEM↩ ORY_PER_BD

where:

- ETH_43_PFE_MAX_TXBD_CNT is sum of all EthCtrlConfigEgressFifoBufTotal option values, if option "Multiple PFE drivers support (master-slave)" is enabled, then additional 4 BDs are added (this is the maximum value from all configuration sets)
- ETH_43_PFE_MAX_RXBD_CNT is sum of all EthCtrlConfigIngressFifoBufTotal option values, if option "Common HIF interface" is not HIF_NOCPY, 72 additional BDs are added, if option "Multiple PFE drivers support (master-slave)" is enabled, then additional 4 BDs are added. This is the maximum value from all configuration sets
- MEMORY_PER_BD is either 16 if option "Common HIF interface" is HIF_NOCPY, or it is 24 otherwise

4.4.3 RX Packet Buffers

These are buffers to be used by HIF to store received Ethernet frames. Rx packet buffers are not allocated if option "Common HIF interface" is HIF_NOCPY. In that case BMU buffers are used instead. Both PFE and the host CPU running the driver need r/w access to this region.

The MemMap file allocates the pool in VAR_CLEARED_UNSPECIFIED_BUF_MEM memory, which is put into ".pfe_buf_mem" section.

Required size of the area depends on configured number of RX buffers, paramter EthCtrlConfigIngressFifoBufTotal:

- RxBuffersMemSize[B] = 1544 $*$ rx_buf_num

where:

- rx_buf_num is sum of all EthCtrlConfigIngressFifoBufTotal option values. If option "Enable errata ER↩ R051211 workaround" is enabled, 72 additional Rx buffers are added. If option "Multiple PFE drivers support (master-slave)" is enabled, then additional 4 buffers are added. This is the maximum value from all configuration sets

4.4.4 TX Packet Buffers

Buffers containing Ethernet frames to be transmitted via HIF. Rx packet buffers are not allocated if option "Common HIF interface" is HIF_NOCPY. In that case BMU buffers are used instead. Both PFE and the host CPU running the driver need r/w access to this region.

The MemMap file allocates the pool in VAR_CLEARED_UNSPECIFIED_BUF_MEM memory, which is put into ".pfe_buf_mem" section.

Size of the pool depends on configured number and size of TX buffers, paramter EthCtrlConfigEgressFifoBufTotal and on EthCtrlConfigEgressFifoBufLenByte.

For each configured egress queue:

- TxBuffersMemSizeQueue[B] = EthCtrlConfigEgressFifoBufTotal $*$ aligned_buffer_size

where:

- aligned_buffer_size parameter EthCtrlConfigEgressFifoBufLenByte increased by 24 and aligned up to 8

The values from all configured egress queues are added together:

- TxBuffersMemSize[B] = sum_all(TxBuffersMemSizeQueue)

Maximum value from all configuration sets is used.

4.4.5 Routing Table

L3 routing table. This memory is allocated by Master driver only. Both PFE and the host CPU running the Master driver need r/w access to this region.

The MemMap file allocates the pool in VAR_CLEARED_UNSPECIFIED_BUF_MEM memory, which is put into ".pfe_buf_mem" section.

The size is determined by the number of entries in the table configured via "Routing Table Hash Size" and "Routing Table Collision Size" options. It can be calculated as:

- RTableMemSize[B] = ("Routing Table Hash Size" + "Routing Table Collision Size") $*$ 128

Note

The Routing Table region must reside within the PFE-accessible range 0x00020000 - 0xBFFFFFFF.

4.4.6 IP Ready Flag

Bit 16 of GENCTRL3 register (address 0x4007CAEC) is used by PFE drivers to signal "IP ready" from Master to Slave driver. Master instance requires r/w access while slave instance needs read only access.

4.4.7 PFE Regions isolation

PFE IP and drivers require access to various SoC resources to ensure normal operation. The S32G SoC allows isolation of various system resources by restricting the access via XRDC. Once enabled, the XRDC should ensure following access policies:

Region name	Scope	Region Base	Length	Driver access		PFE Bus Master Access				
				Master	Slave	HIF0	HIF1	HIF2	HIF3	DDR
PFE registers	System	0x46000000	0x1000000	r/w	-	-	-	-	-	-
LMEM	System	0x46000000	0x20000		-	-	-	-	-	r/w
HIF chnl. 0 registers	System	0x46098100	0x100		r/w)∗	-	-	-	-	-
HIF chnl. 1 registers	System	0x46098200	0x100		r/w)∗	-	-	-	-	-
HIF chnl. 2 registers	System	0x46098300	0x100		r/w)∗	-	-	-	-	-
HIF chnl. 3 registers	System	0x46098400	0x100		r/w)∗	-	-	-	-	-
HIFNOCPY registers	System	0x460D0000	0x100		-	-	-	-	-	-
GENCTRL3	System	0x4007CA↩ EC	0x4	r/w	r	-	-	-	-	-
System buffers	System	CFG	CFG	r/w)∗∗	-	-	-	-	-	r/w
RX buffer desc.	Driver	CFG	CFG	r/w		r	w	-	-	-
TX buffer desc.	Driver	CFG	CFG	r/w		r	w	-	-	-
RX buffers	Driver	CFG	CFG	r/w		-	-	w	-	-
TX buffers	Driver	CFG	CFG	r/w		-	-	-	r	-
Routing table	System	CFG	CFG	r/w	-	-	-	-	-	r/w

)∗ Only if the driver instance uses this channel for RX/TX.
)∗∗ Only if thr driver instance uses the HIFNOCPY channel.
CFG - Depends on PFE driver configuration.
System Scope - Region instantiated at system level.
Driver Scope - Region instantiated per driver instance.

Chapter 5

Function calls to module

5.1 Function Calls during Start-up

Before system clock is initialized through MCU driver, it is necessary to call function *Eth_43_PFE_PreInit*. This is necessary due to hardware limitation to configure physical layer iterfaces (MII mode).

Note

Interface configuration is not applied from driver running in slave mode.

Then PFE clock shall be configured by *Mcu* module and pins of EMAC interfaces shall be configured by *Port* module. If SGMII mode is used for PHY connection, the SERDES shall be configured by Serdes module.

Then the Ethernet driver shall be initialized by function *Eth_43_PFE_Init*.

Each Ethernet controller then can be started by the *Eth_43_PFE_SetControllerMode* function call with *CtrlMode* set to *ETH_MODE_ACTIVE*.

5.2 Function Calls during Shutdown

The application should ensure that all the Ethernet frame transmissions have been completed before the module is shut down by the *Eth_43_PFE_SetControllerMode* function call with *CtrlMode* set to *ETH_MODE_DOWN*. Note that all not transmitted buffers are flushed and all locked transmit buffers are unlocked when the controller is disabled. All receive buffers are also discarded when the controller is stopped so the *Eth_Receive* function should be called before the shut down. To shut-down the driver, call *Eth_43_PFE_DeInit* function.

Note

In master-slave scenarios, the master should not call *Eth_43_PFE_DeInit* when slaves are still running. The recommended sequence is to call *Eth_43_PFE_DeInit* on slaves first, then call *Eth_43_PFE_DeInit* on the master.

5.3 Function Calls during Wake-up

None. Wakeup is curently not supported.

Chapter 6

Module requirements

- Exclusive areas to be defined in BSW scheduler
- Peripheral Hardware Requirements
- Clocking Requirements
- ISR to configure within AutosarOS - dependencies
- ISR Macro
- Other AUTOSAR modules - dependencies
- Data Cache Restrictions
- User Mode support
- Multicore Support

6.1 Exclusive areas to be defined in BSW scheduler

The Ethernet driver is using services of the Schedule Manager (SchM) for entering and exiting critical regions (accessing protected resources). SchM implementation is done by the integrators of the RTD using OS or non-OS services. For testing the Ethernet driver, SchM stubs are used.

Exclusive area matrix is provided in separate decument: AUTOSAR_MCAL_ETH_43_PFE_EXCLUSIVE_A↩ REAS.xlsx

Exclusive areas that are required by the Ethernet driver are listed in the document, as well as mapping of the exclusive areas to driver API.

Note

Exlusive area 99 should not be implemented as suspend all interrupts.

6.2 Peripheral Hardware Requirements

For each used EMAC, Ethernet Transceiver should be connected to the PFE_MAC pins which should be configured to be used by the PFE. EMAC pins need to be configured based on EMAC configuration (MII inteface mode). Configure fast slew-rate for all Ethernet pins, otherwise packet loss may occur.

Pins to be configured in Port plugin for commonly used modes:

- SGMII mode:
  - SGMII pins are dedicated and require no configuration on S32G2 and S32G3 devices.
- RGMII mode:
  - PortPin_PFE_MACx_RXD0
  - PortPin_PFE_MACx_RXD1
  - PortPin_PFE_MACx_RXD2
  - PortPin_PFE_MACx_RXD3
  - PortPin_PFE_MACx_RXDV
  - PortPin_PFE_MACx_RX_CLK
  - PortPin_PFE_MACx_TXD0
  - PortPin_PFE_MACx_TXD1
  - PortPin_PFE_MACx_TXD2
  - PortPin_PFE_MACx_TXD3
  - PortPin_PFE_MACx_TX_CLK
  - PortPin_PFE_MACx_TX_EN
  - PortPin_PFE_MACx_MDC (optional)
  - PortPin_PFE_MACx_MDIO (optional) Where "x" stands for MAC number (0-2).

6.3 Clocking Requirements

6.3.1 PFE Clock Requirements

- PFE_SYS_CLK shall be set to 300 MHz
- PFE_PE_CLK shall be set to 600 MHz

Additional clock requirements if option *EthGlobalTimeSupport* is enabled:

- GMAC_TS_CLK shall be set to 200 MHz
- XBAR_CLK shall be set to 400 MHz - required due to HW constraints

6.3.2 EMAC Clock Requirements

Depending on selected speed and MII inteface mode, it is required to configure coresponding clock (PFE_MAC_n_TX_CLK, PFE_MAC_n_RX_CLK, PFE_MAC_n_EXT_R↩ EF_CLK) in the MCU for each used EMAC.

For MII:

- 10Mbps - Frequency of 2.5MHz must be provided to RX_CLK and TX_CLK.
- 100Mbps - Frequency of 25MHz must be provided to RX_CLK and TX_CLK.

For RMII: For all speeds frequency of 50MHz must be provided to RMII_CLK (PFE_MAC_n_EXT_REF_CLK).

- 10Mbps - Frequency of 2.5MHz must be provided to RX_CLK and TX_CLK.
- 100Mbps - Frequency of 25MHz must be provided to RX_CLK and TX_CLK.

For RGMII:

- 10Mbps - Frequency of 2.5MHz must be provided to RX_CLK and TX_CLK.
- 100Mbps - Frequency of 25MHz must be provided to RX_CLK and TX_CLK.
- 1000Mbps - Frequency of 125MHz must be provided to RX_CLK and TX_CLK.

For SGMII (all baudrates):

- Serdes needs to be configured to enable SGMII mode for selected MAC(s), refer to [15]
- RX_CLK needs to be connected to SERDES_x_XPCS_y_CDR on CGM2_MUX4/5/6
- TX_CLK has different requirements depending on EMAC and derivative:
  - All EMACs on S32G3 and EMAC0 on S32G2: TX_CLK needs to be connected to SERDES_x_XPC↩ S_y_TX in GENCTRL1 mux.
  - EMAC1 and EMAC2 on S32G2: TX_CLK needs to be connected to SERDES_x_XPCS_y_TX on CGM2_MUX2/3.

Here is a summary table:

Device	MAC	Mux	Recommended input
S32G2	EMAC0	S32G_GPR_GENCTRL1 (bit0)	SERDES_1_XPCS_0_TX_CLK
S32G2	EMAC1	CGM2_MUX2	SERDES_1_XPCS_1_TX_CLK
S32G2	EMAC2	CGM2_MUX3	SERDES_0_XPCS_1_TX_CLK
S32G3	EMAC0	SRC_1_GENCTRL1 (bit0)	SERDES_1_XPCS_0_TX_CLK
S32G3	EMAC1	SRC_1_GENCTRL1 (bit1)	SERDES_1_XPCS_1_TX_CLK
S32G3	EMAC2	SRC_1_GENCTRL1 (bit2)	SERDES_0_XPCS_1_TX_CLK

6.4 ISR to configure within AutosarOS - dependencies

The following ISRs are used by the Ethernet driver and they need to be assigned to a priority level and enabled:

- BMU interrupt shall be enabled only if option "Enable BMU Interrupt" is enabled,
- HIF interrupt shall be enabled only if any Rx or Tx interrupt is enabled in Controller configuration. The interrupt is shared between Rx and Tx and among all controllers. Only one HIF ISR is available per PFE MCAL driver, depending on which HIF is used by the driver (configuration option "Common HIF interface").

ISR Name	NVIC Interrupt ID	Used by
Eth_43_PFE_HifIrqHdlr_0	190	HIF0
Eth_43_PFE_HifIrqHdlr_1	191	HIF1
Eth_43_PFE_HifIrqHdlr_2	192	HIF2
Eth_43_PFE_HifIrqHdlr_3	193	HIF3
Eth_43_PFE_BmuIrqHdlr	194	all
Eth_43_PFE_HifNoCpyIrqHdlr	195	HIFNOCPY

6.5 ISR Macro

Ethernet driver uses the ISR macro to define the functions that will process hardware interrupts. Depending on whether the OS is used or not, this macro can have different definitions.

6.5.1 Without an Operating System

The macro USING_OS_AUTOSAROS must not be defined.

6.5.1.1 Using Software Vector Mode

The macro *USE_SW_VECTOR_MODE* must be defined and the ISR macro is defined as:

```
#define ISR(IsrName) void IsrName(void)
```

In this case, the drivers' interrupt handlers are normal C functions and their prologue/epilogue will handle the context save and restore.

6.5.1.2 Using Hardware Vector Mode

The macro *USE_SW_VECTOR_MODE* must not defined and the ISR macro is defined as:

```
#define ISR(IsrName) INTERRUPT_FUNC void IsrName(void)
```

In this case, the drivers' interrupt handlers must also handle the context save and restore.

6.5.2 With an Operating System

Please refer to your OS documentation for description of the ISR macro.

6.6 Other AUTOSAR modules - dependencies

- **Port**: Needed to configure the pins to be used by the Ethernet driver.
- **DET**: Needed for detecting and reporting development errors. The API function used is *Det_ReportError*. The detection can be configured using the *EthDevErrorDetect* configuration parameter
- **DEM**: Needed for detecting and reporting extended production errors.
- **RTE**: Needed for implementing data consistency through exclusive areas.
- **MCU**: Needed to configure the clocks to be used by the Ethernet driver.
- **EcuC**: Needed to retrieve information about post-build variants.
- **Base**: Needed for common files/definitions which are used by all RTD modules.
- **OS**: Needed to define a mapping between EcuC partitions and EcuC core ids when multicore support is enabled.
- **EthIf**: The callbacks *EthIf_RxIndication* and *EthIf_TxConfirmation* are used to notify the upper layer about an Ethernet frame transmission and reception. The callback *EthIf_CtrlModeIndication* is used to notify the upper layer about mode transitions in the controllers (i.e. ACTIVE -> DOWN or DOWN -> ACTIVE)
- **EthTrcv**: The callbacks _EthTrcv_ReadMiiIndication and _EthTrcv_WriteMiiIndication (or their vendor API infix variation) are used to notify the Ethernet transceiver about a successful MDIO transfer.
- **EthSwt**: The callbacks *EthSwt_TxAdaptBufferLengthFunction* and *EthSwt_TxPrepareFrame↩ Function* are used to request the Ethernet switch to do the preparations for a Tx buffer. The callbacks *Eth↩ Swt_TxProcessFrameFunction* and *EthSwt_TxFinishedIndicationFunction* are used to request the Ethernet switch to process the Ethernet frame before transmission. The callbacks *EthSwt_RxProcess↩ FrameFunction* and *EthSwt_RxFinishedIndicationFunction* are used to notify the Ethernet switch of a received Ethernet frame.
- **Serdes**: This module is not directly called from the PFE driver. However, if the PFE driver uses SGMII mode for PHY connection, the application needs to configure the SERDES. **TM**: Ethernet driver uses function *Tm_BusyWait1us32bit* from Time Services module.

6.7 Data Cache Restrictions

The descriptors, data buffers and routing table are placed in a non-cacheable memory sections delimited by:

- *ETH_43_PFE_START_SEC_VAR_CLEARED_UNSPECIFIED_BD_MEM* and *ETH_43_PFE_STOP_SEC_VAR_↩ CLEARED_UNSPECIFIED_BD_MEM*.
- *ETH_43_PFE_START_SEC_VAR_CLEARED_UNSPECIFIED_BUF_MEM* and *ETH_43_PFE_STOP_SEC_VAR↩ _CLEARED_UNSPECIFIED_BUF_MEM*.
- *ETH_43_PFE_START_SEC_VAR_CLEARED_UNSPECIFIED_BMU_MEM* and *ETH_43_PFE_STOP_SEC_VAR↩ _CLEARED_UNSPECIFIED_BMU_MEM*.

As long as the MPU is properly configured to define the memory region containing the aforementioned section as non-cacheable, there should be no data cache coherency problems.

6.8 User Mode support

The Ethernet Driver currently supports running in user mode.

6.9 Multicore Support

The Ethernet Driver implements the "Autosar 4.4 MCAL Multicore Distribution" according to type I, in which the mappable element is set to the Hardware Controller. For additional details, please refer to the "AUTOSAR_EXP↩ _BSWDistributionGuide" document.

The Ethernet Driver is currently implemented as signle mappable element, which can be allocated to zero or one EcuC partition, by means of "EthEcucPartionRef". If the Ethernet Driver is mapped to zero EcuC partitions, the behavior reverts to single-core implementation, similar to previous AUTOSAR versions. Otherwise, if it is mapped to one EcuC partition, then the following multi-core assumptions are enforced:

1. The module assumes that each interrupt is routed by the system only to the core on which it is supposed to be serviced.

To enable multicore support:

1. Add one EcuC partition to the list *EthEcucPartitionRef*.

Chapter 7

Main API Requirements

- Main function calls within BSW scheduler
- API Requirements
- Calls to Notification Functions, Callbacks, Callouts

7.1 Main function calls within BSW scheduler

The Ethernet Driver contains a main function that shall be configured to be scheduled by the BSW Scheduler:

void Eth_43_PFE_MainFunction(void);

This function:

- checks for controller errors and reports them to Health Monitor and to DEM,
- check for lost frames and reports them to DEM,
- polls state changes and calls `EthIf_CtrlModeIndication` when the controller mode is changed,
- if routing table is enabled, it manages time-outs of routing table records,
- if option EthGlobalTimeSupport is enabled, in manages time-outs in timestamp database (in case a timestamp is not received),
- if EthGlobalTimeSupport and Tx interrupts are enabled, it may call EthIf_TxConfirmation for timestamped frames,
- if Rx interrupts are disabled, the main function is polling for IHC packets and executes inter-core calls from other drivers. It is needed for communication between master and slave drivers and for FCI proxy.

7.2 API Requirements

None

7.3 Calls to Notification Functions, Callbacks, Callouts

The Ethernet Driver provides no configurable notification functions, callbacks, or callouts.

7.3.1 Notification functions

The Ethernet Driver does not provide notification functions.

7.3.2 Call-backs

- EthIf_RxIndication: This EthIf interface is called when one or more unread Ethernet frames are waiting in the receive buffers to be processed. Call is made from the Eth_43_PFE_Receive function or from HIF interrupt handler.
- EthIf_TxConfirmation: This EthIf interface is called when one or more Ethernet frames were transmitted and had been set to confirm the transmission. Call is made from the Eth_43_PFE_TxConfirmation function or HIF interrupt handler.
- EthIf_CtrlModeIndication: This EthIf interface is called when a controller mode was changed.
- EthTrcv_ReadMiiIndication: This EthTrcv function is called when previously requested MII read operation is done
- EthTrcv_WriteMiiIndication: This EthTrcv function is called when previously requested MII write operation is done

Note

EthTrcv callback function names may contain configurable vendor infix.

7.3.3 Callouts

The Ethernet Driver does not provide callouts.

Chapter 8

Memory allocation

8.1 Sections to be defined in Eth_43_PFE_MemMap.h

SEC_CONST_UNSPECIFIED

- Section Type: Constants
- Description: Used for constants, constant structures and constant arrays when the SIZE (alignment) does not fit the criteria of 8, 16 or 32 bit. These constants are initialized by start-up code and they can be put into memory with read-only access.
- Start section macro: ETH_43_PFE_START_SEC_CONST_UNSPECIFIED
- End section macro: ETH_43_PFE_STOP_SEC_CONST_UNSPECIFIED

SEC_CONST_16

- Section Type: Constants
- Description: Used for constants which have to be aligned to 16 bits. For instance used for 16-bit constants or used for composite data types (arrays, structs) containing elements of maximum 16 bits. These constants are initialized by start-up code and they can be put into memory with read-only access.
- Start section macro: ETH_43_PFE_START_SEC_CONST_16
- End section macro: ETH_43_PFE_STOP_SEC_CONST_16

SEC_CONST_32

- Section Type: Constants

- Description: Used for constants which have to be aligned to 32 bits. For instance used for 32-bit constants or used for composite data types (arrays, structs) containing elements of maximum 32 bits. These constants are initialized by start-up code and they can be put into memory with read-only access.
- Start section macro: ETH_43_PFE_START_SEC_CONST_32
- End section macro: ETH_43_PFE_STOP_SEC_CONST_32

SEC_CONST_8

- Section Type: Constants
- Description: Used for constants which have to be aligned to 8 bits. For instance used for 8-bit constants or used for composite data types (arrays, structs) containing elements of maximum 8 bits. These constants are initialized by start-up code and they can be put into memory with read-only access.
- Start section macro: ETH_43_PFE_START_SEC_CONST_8
- End section macro: ETH_43_PFE_STOP_SEC_CONST_8

SEC_CONFIG_DATA_UNSPECIFIED

- Section Type: Configuration Data
- Description: Used for configuration data
- Start section macro: ETH_43_PFE_START_SEC_CONFIG_DATA_UNSPECIFIED
- End section macro: ETH_43_PFE_STOP_SEC_CONFIG_DATA_UNSPECIFIED

SEC_VAR_INIT_UNSPECIFIED

- Section Type: Variables
- Description: Used for variables, structures, arrays when the SIZE (alignment) does not fit the criteria of 8, 16 or 32 bit. These variables are initialized by start-up code.
- Start section macro: ETH_43_PFE_START_SEC_VAR_INIT_UNSPECIFIED
- End section macro: ETH_43_PFE_STOP_SEC_VAR_INIT_UNSPECIFIED

SEC_VAR_INIT_32

- Section Type: Variables
- Description: Used for variables which have to be aligned to 32 bits. For instance used for 32-bit variables or used for composite data types (arrays, structs) containing elements of maximum 32 bits. These variables are initialized by start-up code.
- Start section macro: ETH_43_PFE_START_SEC_VAR_INIT_32
- End section macro: ETH_43_PFE_STOP_SEC_VAR_INIT_32

SEC_VAR_INIT_8

- Section Type: Variables
- Description: Used for variables which don't need to be aligned. For instance used for 8-bit variables or used for composite data types (arrays, structs) containing elements of maximum 8 bits. These variables are initialized by start-up code.
- Start section macro: ETH_43_PFE_START_SEC_VAR_INIT_8
- End section macro: ETH_43_PFE_STOP_SEC_VAR_INIT_8

SEC_VAR_INIT_BOOLEAN

- Section Type: Variables
- Description: Used for variables of type boolean, which don't need to be aligned. These variables are initialized by start-up code.
- Start section macro: ETH_43_PFE_START_SEC_VAR_INIT_BOOLEAN
- End section macro: ETH_43_PFE_STOP_SEC_VAR_INIT_BOOLEAN

SEC_CODE

- Section Type: Code
- Description: Used for code
- Start section macro: ETH_43_PFE_START_SEC_CODE
- End section macro: ETH_43_PFE_STOP_SEC_CODE

SEC_VAR_CLEARED_UNSPECIFIED

- Section Type: Variables
- Description: Used for variables, structures, arrays when the SIZE (alignment) does not fit the criteria of 8, 16 or 32 bit. These variables must be cleared by start-up code.
- Start section macro: ETH_43_PFE_START_SEC_VAR_CLEARED_UNSPECIFIED
- End section macro: ETH_43_PFE_STOP_SEC_VAR_CLEARED_UNSPECIFIED

SEC_VAR_CLEARED_16

- Section Type: Variables
- Description: Used for variables which have to be aligned to 16 bits. For instance used for 16-bit variables or used for composite data types (arrays, structs) containing elements of maximum 16 bits. These variables must be cleared by start-up code.
- Start section macro: ETH_43_PFE_START_SEC_VAR_CLEARED_16
- End section macro: ETH_43_PFE_STOP_SEC_VAR_CLEARED_16

SEC_VAR_CLEARED_32

- Section Type: Variables
- Description: Used for variables which have to be aligned to 32 bits. For instance used for 32-bit variables or used for composite data types (arrays, structs) containing elements of maximum 32 bits. These variables must be cleared by start-up code.
- Start section macro: ETH_43_PFE_START_SEC_VAR_CLEARED_32
- End section macro: ETH_43_PFE_STOP_SEC_VAR_CLEARED_32

SEC_VAR_CLEARED_8

- Section Type: Variables
- Description: Used for variables which don't need to be aligned. For instance used for 8-bit variables or used for composite data types (arrays, structs) containing elements of maximum 8 bits. These variables must be cleared by start-up code.
- Start section macro: ETH_43_PFE_START_SEC_VAR_CLEARED_8
- End section macro: ETH_43_PFE_STOP_SEC_VAR_CLEARED_8

SEC_VAR_CLEARED_UNSPECIFIED_BD_MEM

- Section Type: Variables
- Description: This section is used to store buffer descriptors accessed by PFE driver and HIF bus masters. This section does not need to be cleared by start-up code. This section must be cache inhibited.
- Start section macro: ETH_43_PFE_START_SEC_VAR_CLEARED_UNSPECIFIED_BD_MEM
- End section macro: ETH_43_PFE_STOP_SEC_VAR_CLEARED_UNSPECIFIED_BD_MEM

SEC_VAR_CLEARED_UNSPECIFIED_BUF_MEM

- Section Type: Variables
- Description: This section is used to store Rx and Tx buffers accessed by PFE driver and HIF bus masters and to store routing table. This section does not need to be cleared by start-up code. This section must be cache inhibited.
- Start section macro: ETH_43_PFE_START_SEC_VAR_CLEARED_UNSPECIFIED_BUF_MEM
- End section macro: ETH_43_PFE_STOP_SEC_VAR_CLEARED_UNSPECIFIED_BUF_MEM

SEC_VAR_CLEARED_UNSPECIFIED_BMU_MEM

- Section Type: Variables
- Description: This section is used to store buffers managed by BMU (part of PFE). Refer to section "PFE System Buffers" for specific requirements for this section. This section must be cleared by start-up code. This section must be cache inhibited.
- Start section macro: ETH_43_PFE_START_SEC_VAR_CLEARED_UNSPECIFIED_BMU_MEM
- End section macro: ETH_43_PFE_STOP_SEC_VAR_CLEARED_UNSPECIFIED_BMU_MEM

8.2 Linker command file

Memory shall be allocated for every section defined in the driver's <Module>_MemMap.h.

If PFE Class firmware file s32g_pfe_class.c is being built with PFE driver, then memory shall be allocated also for every section defined in the file. For more details refer to section The PFE Class Firmware Integration

Chapter 9

Integration Steps

- The PFE Class Firmware Integration

This section gives a brief overview of the steps needed for integrating this module:

1. Get and integrate PFE Class firmware. For more details refer to section The PFE Class Firmware Integration
2. Add the PFE module to configuration project, configure it and generate the required module configuration files. For more details refer to section Files Required for Compilation
3. Make sure that a memory map file for the PFE module is present in build project and that it contains all required memory sections. For more details refer to section Sections to be defined in Eth_43_PFE_MemMap.h.
4. Make sure that linker command file properly places in memory all memory sections from Eth_43_PFE_↩ MemMap.h and also all memory section defined in PFE Class firmware (if the firmware is being compiled into the application).
5. Compile & build the module with all the dependent modules. For more details refer to section Building the Driver

9.1 The PFE Class Firmware Integration

9.1.1 Getting the Firmware The firmware can be downloaded from nxp.com: log in, click on "My NXP Account", click on "Software Licensing and Support", click at "View accounts" (Software accounts). There either "Product List" or "Product search" can be used to to find "S32G PFE Firmware".

The driver compatibility is not limited to single firmware version. At least firmware version specified in PFE MCAL driver release notes is compatible.

Note

Driver is checking firmware compatibility on startup. Function Eth_43_PFE_Init fails if incompatible firmware is integrated.

9.1.2 Deploying the Firmware PFE MCAL driver provides flexibity in firmware integration. There are two ways to deploy the firmware, choose and follow one of them:

9.1.2.1 Building the firmware binary into the application

This is a simple way to integrate the firmware and it is also demonstrated in provided example application.

1. In PFE module configuration set option "Class firmware load from" to value **PfeClassFwArrayLabel** and in option PfeClassFwArrayLabel type the name of array containing the firmware binary (or leave it at default value).
2. In downloaded firmware package find files s32g_pfe_class.c and s32g_pfe_class.h and them into build project.
3. Make sure that memory section .pfe_class_fw_mem, which is defined in file s32g_pfe_class.c, is properly managed by the linker command file.

9.1.2.2 Deploying the firmware binary in memory by other means

This is a more flexible way to integrate the firmware, which also allows firmware to be updated without rebuilding the application.

1. In downloaded firmware package find file s32g_pfe_class.fw. Note the size of the file.
2. Reserve a chunk of memory to store the firmware binary in. Make sure the size of the memory is large enough (at least the size of the file). If future firmware updates are expected, the memory size should be larger (for future firmware size variations).
3. In PFE module configuration set option "Class firmware load from" to value **PfeClassFwMemoryAddress** and in option **PfeClassFwMemoryAddress** enter the address at which the firmware binary will be loaded.
4. Make sure that the content of file s32g_pfe_class.fw is loaded on address specified in configuration. It shall be loaded as it is, without any conversion or parsing.

Note

The firmware binary cannot be modified, updated or overwritten while the master PFE driver is being initialized by function Eth_43_PFE_Init. During runtime there is no access to firmware binary from driver and memory can be reused by application.

Note that during restart of master driver, function Eth_43_PFE_Init is being called again and it will need access to firmware binary again. (If the binary was previously overwritten and its memory reused, the binary must be restored before every call of Eth_43_PFE_Init (on master driver))

Chapter 10

External assumptions for driver

The section presents requirements that must be complied with when integrating the PFE Ethernet Driver into the application.

[SWS_Eth_00120]

<< API function - Header File - Description -
Det_ReportError - Det.h - Service to report development errors. -
EthSwt_EthRxFinishedIndication - EthSwt_Eth.h - Indication for a finished receive process for a specific Ethernet frame, which results in providing the management information retrieved during EthSwt_EthRxProcessFrame(). -
EthSwt_EthRxProcessFrame - EthSwt_Eth.h - Function inspects the Ethernet frame passed by the data pointer for management information and stores it for later use in EthSwt_EthRxFinishedIndication(). -
EthSwt_EthTxAdaptBufferLength - EthSwt_Eth.h - Modifies the buffer length to be able to insert management information. -
EthSwt_EthTxFinishedIndication - EthSwt_Eth.h - Indication for a finished transmit process for a specific Ethernet frame. -
EthSwt_EthTxPrepareFrame - EthSwt_Eth.h - Prepares the Ethernet frame for common Ethernet communication (frame shall be handled by switch according to the common address resolution behavior) and stores the information for processing of EthSwt_EthTxFinishedIndication(). -
EthSwt_EthTxProcessFrame - EthSwt_Eth.h - Function inserts management information into the Ethernet frame. ->>

Note

Under control of EthSwt module

[SWS_Eth_00158]

<< Name: - Eth_ModeType -
Type: - Enumeration -
Range: - ETH_MODE_DOWN - 0x00 - Controller disabled -
ETH_MODE_ACTIVE - 0x01 - Controller enabled -
Description: - This type defines the controller modes -
Available via: - Eth_GeneralTypes.h ->>

Note

Under control of BASE module

[SWS_Eth_00159]

<< Name: - Eth_StateType -
Type: - Enumeration -
Range: - ETH_STATE_UNINIT - 0x00 - Driver is not yet configured -
ETH_STATE_INIT - 0x01 - Driver is configured -
Description: - Status supervision used for Development Error Detection. The state shall be available for debugging. -
Available via: - Eth_GeneralTypes.h ->>

Note

Under control of BASE module

[SWS_Eth_00160]

<< Name: - Eth_FrameType -
Type: - uint16 -
Description: - This type defines the Ethernet frame type used in the Ethernet frame header -
Available via: - Eth_GeneralTypes.h ->>

Note

Under control of BASE module

[SWS_Eth_00161]

<< Name: - Eth_DataType -
Type: - uint8, uint16, uint32 -
Description: - This type defines the Ethernet data type used for data transmission. Its definition depends on the used CPU. -
Available via: - Eth_GeneralTypes.h ->>

Note

Under control of BASE module

[SWS_Eth_00162]

<< Name: - Eth_RxStatusType -
Type: - Enumeration -
Range: - ETH_RECEIVED - 0x00 - Ethernet frame has been received, no further frames available -
ETH_NOT_RECEIVED - 0x01 - Ethernet frame has not been received, no further frames available -
ETH_RECEIVED_MORE_DATA_AVAILABLE - 0x02 - Ethernet frame has been received, more frames are available -
Description: - Used as out parameter in Eth_Receive() indicates whether a frame has been received and if so, whether more frames are available or frames got lost. -
Available via: - Eth_GeneralTypes.h ->>

External assumptions for driver

Note

Under control of BASE module

[SWS_Eth_00163]

<< Name: - Eth_FilterActionType -
Type: - Enumeration -
Range: - ETH_ADD_TO_FILTER - 0x00 - add the MAC address to the filter, meaning allow reception -
ETH_REMOVE_FROM_FILTER - 0x01 - remove the MAC address from the filter, meaning reception is blocked in the lower layer -
Description: - The Enumeration Type Eth_FilterActionType describes the action to be taklen for the MAC address given in ∗PhysAddrPtr. -
Available via: - Eth_GeneralTypes.h ->>

Note

Under control of BASE module

[SWS_Eth_00175]

<< Name: - Eth_BufIdxType -
Type: - uint32 -
Description: - Ethernet buffer identifier type. -
Available via: - Eth_GeneralTypes.h ->>

Note

Under control of BASE module

[SWS_Eth_00177]

<< Name: - Eth_TimeStampQualType -
Type: - – -
Range: - ETH_VALID - 0 - – -
ETH_INVALID - 1 - – -
ETH_UNCERTAIN - 2 - – -
Description: - Depending on the HW, quality information regarding the evaluated time stamp might be supported. If not supported, the value shall be always Valid. For Uncertain and Invalid values, the upper layer shall discard the time stamp. -
Available via: - Eth_GeneralTypes.h ->>

Note

Under control of BASE module

[SWS_Eth_00178]

<< Name: - Eth_TimeStampType -
Type: - Structure -
Element: - uint32 - nanoseconds - Nanoseconds part of the time -
uint32 - seconds - 32 bit LSB of the 48 bits Seconds part of the time -
uint16 - secondsHi - 16 bit MSB of the 48 bits Seconds part of the time -
Description: - Variables of this type are used for expressing time stamps including relative time and absolute calendar time. The absolute time starts at 1970-01-01.
0 to 281474976710655s
== 3257812230d
[0xFFFF FFFF FFFF]
0 to 999999999ns
[0x3B9A C9FF]
invalid value in nanoseconds: [0x3B9A CA00] to [0x3FFF FFFF]
Bit 30 and 31 reserved, default: 0 -
Available via: - Eth_GeneralTypes.h ->>

Note

Under control of BASE module

[SWS_Eth_00179]

<< Name: - Eth_TimeIntDiffType -
Type: - Structure -
Element: - Eth_TimeStampType - diff - time difference -
boolean - sign - Positive (True) / negative (False) time -
Description: - Variables of this type are used to express time differences. -
Available via: - Eth_GeneralTypes.h ->>

Note

Under control of BASE module

[SWS_Eth_00180]

<< Name: - Eth_RateRatioType -
Type: - Structure -
Element: - Eth_TimeIntDiffType - IngressTimeStampDelta - IngressTimeStampSync2 - IngressTimeStampSync1 - Eth_TimeIntDiffType - OriginTimeStampDelta - OriginTimeStampSync2[FUP2] - OriginTimeStampSync1[FUP1] -
Description: - Variables of this type are used to express frequency ratios. -
Available via: - Eth_GeneralTypes.h ->>

Note

Under control of BASE module

[CPR_RTD_00210]

<< A configuration parameter shall control that header files of other modules affecting version checks will not be performed, respectively shall be performed. -
Per default those version checks shall not be executed. -
Note: The configuration parameter is "DISABLE_MCAL_INTERMODULE_ASR_CHECK" ->>

Note

Define macro DISABLE_MCAL_INTERMODULE_ASR_CHECK to disable AUTOSAR version check between modules

How to Reach Us:

Home Page:
nxp.com

Web Support:
nxp.com/support

Information in this document is provided solely to enable system and software implementers to use NXP products. There are no express or implied copyright licenses granted hereunder to design or fabricate any integrated circuits based on the information in this document. NXP reserves the right to make changes without further notice to any products herein.

NXP makes no warranty, representation, or guarantee regarding the suitability of its products for any particular purpose, nor does NXP assume any liability arising out of the application or use of any product or circuit, and specifically disclaims any and all liability, including without limitation consequential or incidental damages. "Typical" parameters that may be provided in NXP data sheets and/or specifications can and do vary in different applications, and actual performance may vary over time. All operating parameters, including "typicals," must be validated for each customer application by customer's technical experts. NXP does not convey any license under its patent rights nor the rights of others. NXP sells products pursuant to standard terms and conditions of sale, which can be found at the following address: nxp.com/SalesTermsandConditions.